

# State & Props Lab (Practical)

Welcome to the practical lab for State & Props! In this session, you will transform your static HTML form into an interactive, state-driven React component using the `useState` hook, and pass data between components using `props`.

## 1. Concept Overview

In this lab, you will:

- Implement **controlled components** where form input values are synchronized directly with React state.
- Use the `useState` hook to capture and store user input features in real-time.
- Pass state data down to a child display component using `props`.

## 2. Step-by-Step Lab Instructions

### Step 1: Update `PredictionForm.jsx` to Use State

Open your project in **Visual Studio Code** and update

`src/components/PredictionForm.jsx` to manage input values using `useState`:

```
// src/components/PredictionForm.jsx
import { useState } from "react";
import "./PredictionForm.css";

function PredictionForm() {
  // Declare state variables for input features and prediction results
  const [feature1, setFeature1] = useState("");
  const [feature2, setFeature2] = useState("");
  const [submittedData, setSubmittedData] = useState(null);

  const handleSubmit = (e) => {
    e.preventDefault();
    // Store submitted values to pass down via props or display
    setSubmittedData({
      f1: feature1,
```

```

        f2: feature2,
    });
    console.log("Form Submitted:", { feature1, feature2 });
};

return (
    <main className="form-container">
        <section className="card">
            <h2>Model Feature Input</h2>
            <form onSubmit={handleSubmit}>
                <div className="input-group">
                    <label htmlFor="feature1">Primary Feature (e.g., Sq Ft /
Hours):</label>
                    <input
                        type="number"
                        id="feature1"
                        value={feature1}
                        onChange={(e) => setFeature1(e.target.value)}
                        placeholder="Enter value..."
                        required
                    />
                </div>

                <div className="input-group">
                    <label htmlFor="feature2">Secondary Feature (e.g., Rooms /
Age):</label>
                    <input
                        type="number"
                        id="feature2"
                        value={feature2}
                        onChange={(e) => setFeature2(e.target.value)}
                        placeholder="Enter value..."
                        required
                    />
                </div>

                <button type="submit" className="submit-btn">Run
Prediction</button>
            </form>

            {/* Conditionally render results if data has been submitted */}
            {submittedData && (
                <div className="result-box">
                    <h3>Captured Inputs:</h3>
                    <p>Feature 1: {submittedData.f1}</p>
                    <p>Feature 2: {submittedData.f2}</p>
                    <p className="status-pending">Ready for FastAPI Backend
Integration!</p>
                </div>
            )}
        </section>
    </main>

```

```
);  
}
```

```
export default PredictionForm;
```

## Step 2: Add Result Styling to PredictionForm.css

Open `src/components/PredictionForm.css` and append styling for the results box at the bottom:

```
/* Add to bottom of src/components/PredictionForm.css */  
.result-box {  
  margin-top: 1.5rem;  
  padding: 1rem;  
  background-color: #f1f5f9;  
  border-radius: 6px;  
  border-left: 4px solid #3b82f6;  
}  
  
.result-box h3 {  
  margin: 0 0 0.5rem 0;  
  font-size: 1rem;  
  color: #1e293b;  
}  
  
.result-box p {  
  margin: 0.25rem 0;  
  font-size: 0.9rem;  
  color: #475569;  
}  
  
.status-pending {  
  margin-top: 0.75rem !important;  
  font-weight: 600;  
  color: #2563eb !important;  
  font-size: 0.85rem !important;  
}
```

## Step 3: Test Your Interactive Component

1. Make sure your development server is running ( `npm run dev` ).
2. Open your browser ( `http://localhost:5173` ).
3. Type numbers into both input fields and click **Run Prediction**. Notice how the state updates instantly and renders your submitted values right on the card!

---

## 3. Reference Links & Further Reading

- [React useState Documentation](#): Detailed API guide for managing state hooks.

- **Responding to Events in React:** Learn how to handle form submissions and user interactions cleanly.
-